LAPORAN PRAKTIKUM DEVSECOPS

OBSERVABILITY STACK DI DOCKER: CENTRALIZED LOGGING FLUENT BIT DAN POSTGRESQL PADA SISTEM PENERIMAAN MAHASISWA BARU (PMB)

(PRAKTIKUM BAB 6 - STUDI KASUS APLIKASI MULTI-TIER)



Disusun Oleh:

Muhamad Reza Muktasib
3126640060
D4 RPL IT B

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada arsitektur sistem perangkat lunak modern yang berbasis kontainer dan *microservices*, aplikasi tidak lagi berjalan sebagai monolit tunggal yang terisolasi. Beban kerja dipecah menjadi layanan-layanan independen yang berkomunikasi melalui jaringan internal dan memiliki siklus hidup yang bersifat dinamis serta fana (*ephemeral*). Karakteristik ini menimbulkan kompleksitas baru dalam visibilitas operasional: ketika sebuah transaction pengguna mengalami kegagalan, pemeriksaan sederhana terhadap status kontainer (misalnya status `Up` pada perintah `docker compose ps`) sama sekali tidak mencerminkan apakah fungsi bisnis aplikasi berjalan sehat atau sedang mengalami malafungsi internal. Apabila seluruh event records (*log*) hanya tertahan pada *writable layer* masing-masing kontainer, proses investigasi insiden (*incident response*), penelusuran akar masalah (*root cause analysis*), dan rekonstruksi kronologi kejadian menjadi sangat lambat dan berisiko kehilangan data penting saat kontainer dimatikan atau dibuat ulang.

Untuk menjawab tantangan nyata tersebut, praktikum Bab 6 ini mengimplementasikan studi kasus nyata berskala *multi-tier*: **PMB DevSecOps Laboratory** (Sistem Penerimaan Mahasiswa Baru). Laboratorium ini menyimulasikan transaction pendaftaran mahasiswa yang melibatkan interaksi terkoordinasi antara tiga layanan aplikasi Python/Flask (`registration-api`, `auth-service`, dan `verification-worker`) dengan pemisahan mutlak antara basis data transactional (`app-db`) dan basis data pengumpulan telemetri terpusat (`log-db`). seluruh records aliran log aplikasi ditangkap langsung dari `stdout` menggunakan Docker `fluentd` *logging driver*, disalurkan ke engine Fluent Bit v4.2 melalui protokol Forward, dilindungi dengan mekanisme *disk buffering* lokal untuk mencegah *data loss* akibat *backpressure*, lalu disimpan ke dalam basis data PostgreSQL dalam format kolom biner `JSONB` yang diperkuat indeks GIN (*Generalized Inverted Index*).

Melalui arsitektur ini, mahasiswa memosisikan diri sebagai tim operasi dan rekayasa keamanan (*DevSecOps engineers*). Praktikum ini mendemonstrasikan bagaimana korelasi jejak terdistribusi diwujudkan menggunakan identifier unik `request_id`, bagaimana kebijakan *Data Minimization* dan sanitasi informasi sensitif (PII) ditegakkan agar kredensial atau kata sandi tidak bocor ke repositori log, bagaimana investigasi insiden diselesaikan secara presisi menggunakan query SQL analitik pada saat simulasi *dependency timeout*, serta pembuktian nyata ketahanan antrean buffer saat backend penyimpanan log mengalami gangguan sementara.

1.2 Capaian dan Tujuan Praktikum

Berdasarkan silabus dan panduan operasional PMB DevSecOps Laboratory, capaian pembelajaran praktikum ini dirumuskan dalam enam kompetensi utama berikut.

1. Membedakan secara arsitektural dan fungsional antara basis data transactional aplikasi (`app-db`) dengan backend *centralized logging* (`log-db`).
2. Mengonfigurasi dan menghasilkan *structured logging* berbasis JSON dari aplikasi multi-service tanpa membocorkan data pribadi mahasiswa (PII) maupun kredensial otentikasi.
3. Menerapkan mekanisme pelacakan terdistribusi (*distributed tracing*) dengan mengorelasikan event records lintas layanan menggunakan identifier unik `request_id`.
4. Merekonstruksi kronologi insiden operasional secara deterministik menggunakan query SQL analitik pada tabel log terstruktur.
5. Menguji dan menganalisis perilaku *retry*, *dependency timeout*, *filesystem buffering*, *recovery*, serta batas toleransi kehilangan log saat terjadi kegagalan sistem.
6. Mengumpulkan bukti audit (*evidence collection*) yang valid, terukur, dan dapat diverifikasi integritasnya menggunakan *checksum* kriptografis tanpa menyertakan berkas rahasia (*secret*).

BAB II
DASAR TEORI

2.1 Arsitektur Multi-Service PMB dan Docker Fluentd Logging Driver

Dalam metodologi *Twelve-Factor App*, aplikasi cloud-native tidak boleh dibebani dengan pengelolaan penulisan berkas log lokal ke disk. Setiap proses aplikasi wajib meng-emit seluruh log record ke aliran standar `stdout` dan `stderr` tanpa pembungkusan (*unbuffered*). Runtime Docker Engine bertindak sebagai penangkap aliran tersebut melalui mekanisme *logging driver*. Pada laboratorium PMB ini, driver dikonfigurasi ke mode `fluentd` dengan konfigurasi blok ekstensi deklaratif YAML berikut.

Parameter Konfigurasi	Nilai Konfigurasi	Fungsi dan Penjelasan Teknis
`logging.driver`	`fluentd`	Menginstruksikan Docker daemon untuk mengirim log via protokol Forward binary
`fluentd-address`	`127.0.0.1:24224`	Alamat port host tempat engine Fluent Bit mendengarkan input forward
`fluentd-async`	`"true"`	Mode non-blocking agar kontainer aplikasi tidak tertahan saat collector lambat
`fluentd-buffer-limit`	`"4096"`	Batas kapasitas ring buffer memori internal Docker daemon untuk tiap kontainer
`tag`	`pmb.{{.Name}}`	Pola label routing dinamis yang menyertakan nama kontainer sumber

Penerapan opsi `fluentd-async: "true"` merupakan kontrol ketersediaan (*availability control*) yang sangat krusial. Jika mode *asynchronous* tidak diaktifkan (mode default *blocking*), ketika Fluent Bit mengalami lonjakan beban atau berhenti sementara, proses penulisan ke `stdout` oleh aplikasi Flask atau Gunicorn akan mengalami *backpressure* dan tertahan (*hung*), yang pada akhirnya melumpuhkan seluruh portal pendaftaran mahasiswa.

2.2 Topologi Dual Database: Transactional (app-db) vs Logging (log-db)

Salah satu kesalahan mendasar dalam perancangan infrastruktur cloud adalah mencampurkan data transaction bisnis dengan data telemetri log ke dalam satu instans basis data yang sama. Laboratorium PMB menegakkan pemisahan mutlak (*separation of concerns*) melalui topologi dua basis data PostgreSQL independen berikut.

Parameter Pembeda	Application Database (`app-db`)	Centralized Log Database (`log-db`)
Tujuan Utama	Menyimpan status transaction bisnis pendaftaran PMB	Menyimpan jejak audit telemetri kejadian sistem
Model Operasi Data	*Read-heavy* dan *Update-intensive* (ACID Transactions)	*Append-only* ingest berkecepatan tinggi
Skema Tabel Utama	Tabel `registrations` (kolom relasional terstruktur kaku)	Tabel `logs.fluentbit` (kolom `tag`, `time`, `data JSONB`)
Karakteristik Indeks	B-Tree pada `user_ref`, `status`, dan `request_id`	B-Tree pada `time DESC` dan GIN pada kolom `data`
Dampak Kegagalan	Jika down, aplikasi tidak dapat memproses pendaftaran	Jika down, transaction tetap jalan karena log dibuffer
Kebijakan Akses	Diberikan kepada akun aplikasi PMB (`app_user`)	Diberikan kepada akun log shipper (`fb_user`)

Penyimpanan log pada `log-db` memanfaatkan tipe data `JSONB` (*Binary JSON*). Format biner terurai ini memungkinkan PostgreSQL mengekstrak atribut bersarang (seperti `data ->> 'service'`, `data ->> 'severity'`, atau `(data ->> 'log')::jsonb ->> 'error_code'`) dan memanfaatkan indeks GIN (*Generalized Inverted Index*) untuk mengeksekusi query analitik secara instan tanpa melakukan pemindaian menyeluruh (*full-table scan*).

2.3 Mekanisme Distributed Tracing berbasis Correlation ID (request_id)

Pada arsitektur terdistribusi, satu permintaan pengguna melewati berbagai batas layanan (*trust boundaries*): mulai dari browser mahasiswa, diterima oleh `registration-api`, diverifikasi oleh `auth-service`, disimpan di `app-db`, hingga diproses secara asinkron oleh `verification-worker`. Tanpa penanda korelasi, baris log error pada worker tidak dapat dihubungkan dengan transaction pendaftaran tertentu di API.

Sistem PMB menyelesaikan tantangan ini melalui injeksi header HTTP standar `X-Request-ID`. Setiap kali request masuk ke portal, sistem menghasilkan token unik berformat `REQ-PMB-<timestamp>-<random>` (misalnya `REQ-PMB-20261027-144310-9421`). Token ini disuntikkan ke dalam respon HTTP, dicatat pada kolom tabel transaction di `app-db`, serta disematkan ke dalam setiap baris log records terstruktur yang di-emit oleh seluruh layanan. Dengan demikian, teknisi operasi dapat merekonstruksi transaction lifecycle secara utuh (*end-to-end tracing*) hanya dengan memfilter satu nilai `request_id`.

2.4 Keamanan Data: Data Minimization dan PII Sanitization

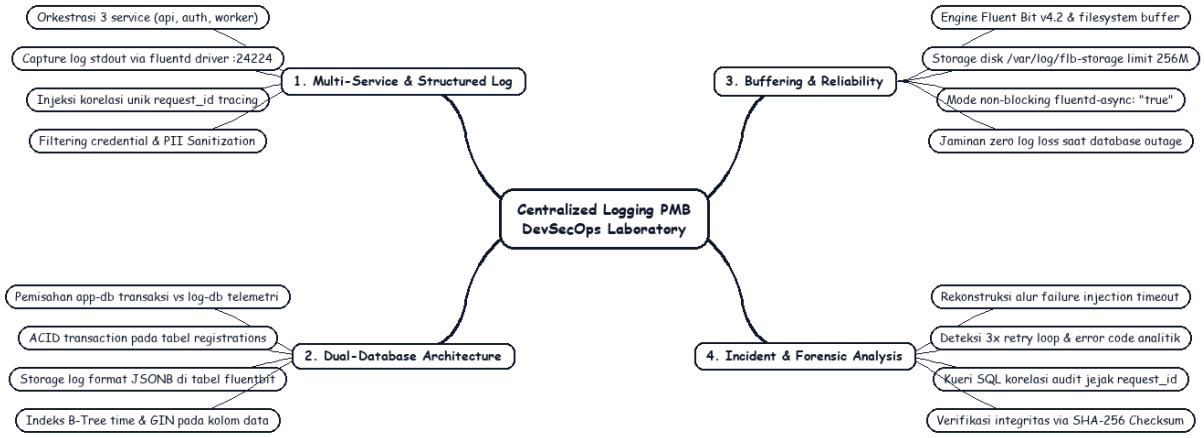
Prinsip keamanan *Data Minimization* (Pasal 5 GDPR dan UU PDP) mewajibkan sistem untuk hanya mengumpulkan dan menyimpan data telemetri yang benar-benar dibutuhkan untuk keperluan operasional dan keamanan. log records sering kali menjadi celah kebocoran data (*data leak*) terbesar di industri karena pengembang secara ceroboh mencetak seluruh *payload* request ke `stdout`.

Pada modul aplikasi PMB (`app/common.py`), diterapkan lapisan sanitasi otomatis (*PII Sanitization*):

1. Kata sandi pengguna (*password*) dan *session token* di-filter dan di-sanitize sehingga tidak pernah dimasukkan ke dalam format structured log.
2. Informasi identitas pribadi seperti Nama Lengkap mahasiswa dan Alamat Email hanya disimpan secara aman di tabel `app-db` yang terenkripsi dan berhak akses terbatas, namun **sengaja tidak dicatat ke dalam log terstruktur Fluent Bit**.
3. Log hanya mencatat identifier bisnis (`registration_id`), kode status pendaftaran, durasi eksekusi, serta kode kesalahan teknis (`error_code`). Hal ini menjamin bahwa jika repositori log berhasil diakses oleh pihak internal atau analis data, tidak ada informasi pribadi mahasiswa yang terekspos.

2.5 Peta Konsep: Centralized Logging PMB DevSecOps Laboratory

Struktur arsitektur pengumpulan log, keterhubungan multi-service, pemisahan basis data, dan alur investigasi insiden PMB dipetakan pada diagram *Mind Map* berikut.



Berdasarkan pemetaan konsep di atas, arsitektur *Centralized Logging* PMB ditopang oleh empat pilar utama berikut.

1. **Multi-Service dan Structured Log** mengorkestrasi tiga komponen aplikasi (`registration-api`, `auth-service`, dan `verification-worker`) yang menuliskan log berformat JSON terstruktur ke `stdout`, dilengkapi penanda korelasi `request_id` dan penyaringan PII otomatis.
2. **Dual-Database Architecture** memisahkan secara fisik basis data transactional ACID `app-db` dari basis data telemetri `log-db` berformat `JSONB` yang dilengkapi indeks inversi tergeneralisasi GIN.
3. **Buffering dan Reliability** menyediakan jaminan keutuhan data (*zero log loss*) melalui engine Fluent Bit v4.2 dengan alokasi buffer penyimpanan fisik di disk `/var/log/flb-storage` bertipe `filesystem` dan batas kapasitas *backpressure* 256MB.
4. **Incident Investigation dan Forensic** memfasilitasi rekonstruksi kronologi transaction secara deterministik menggunakan query SQL analitik pada saat simulasi *dependency timeout*, audit keamanan kebocoran kredensial, serta pembuktian integritas artefak via SHA-256.

BAB III
METODOLOGI DAN LANGKAH KERJA

3.1 Arsitektur Laboratorium PMB dan Rincian Komponen Stack

Infrastruktur laboratorium dibangun menggunakan Docker Compose v2 yang mengisolasi beban kerja ke dalam dua jaringan jembatan (*bridge network*): `app-net` untuk komunikasi transaction aplikasi dan `logging-net` untuk jalur pengumpulan telemetri.

Nama Layanan	Citra Basis / Build	Port Host	Jaringan	Fungsi Arsitektural
`registration-api`	Build `./app` (Python/Flask)	`127.0.0.1:8080`	`app-net`	Menampilkan antarmuka Web UI, manajemen sesi, dan endpoint pendaftaran
`auth-service`	Build `./app` (Gunicorn/Flask)	*Internal only*	`app-net`	Menangani verifikasi autentikasi kredensial pengguna lab
`verification-worker`	Build `./app` (Python script)	*Internal only*	`app-net`	Memproses verifikasi berkas secara asinkron dan simulasi retry
`app-db`	`postgres:16-alpine`	*Internal only*	`app-net`	Menyimpan data transaction relasional tabel `registrations`
`fluent-bit`	`fluent/fluent-bit:4.2.2`	`127.0.0.1:24224`, `2020`	`logging-net`	Mengumpulkan, menampung buffer di disk, dan merutekan log
`log-db`	`postgres:16-alpine`	*Internal only*	`logging-net`	Menyimpan log records terpusat pada tabel `logs.fluentbit` (JSONB)

3.2 Skenario Pengujian: Alur Normal vs Failure Injection Timeout

Pengujian laboratorium dibagi menjadi dua skenario pengujian fungsional utama.

1. **Skenario 1 (Alur Transaction Normal)**
   Mode insiden disetel ke `none`. Mahasiswa mengirim formulir pendaftaran. Alur yang diharapkan: autentikasi berhasil (`login_success`), formulir tersimpan (`registration_submitted`), worker memulai verifikasi (`verification_started`), verifikasi selesai (`verification_completed`), dan status transaction di `app-db` menjadi `VERIFIED`.
2. **Skenario 2 (Simulasi Insiden Dependency Timeout)**
   Mode insiden disetel ke `timeout`. Mahasiswa mengirim pendaftaran baru. Worker mengalami keterlambatan dependensi eksternal, melakukan percobaan ulang hingga 3 kali (`verification_retry`), dan akhirnya memicu batas waktu (`dependency_timeout`). Status akhir transaction menjadi `FAILED` dengan kode kesalahan `VERIFY-TIMEOUT`. Mahasiswa kemudian bertindak sebagai analis forensik untuk membuktikan akar masalah melalui query SQL pada `log-db`.

3.3 Urutan Langkah Kerja Praktikum

Rangkaian prosedur praktikum dilaksanakan dalam langkah-langkah terstandar berikut.

1. Menyiapkan repositori dan menjalankan skrip otomatisasi inisialisasi lingkungan `./scripts/setup.sh` untuk menghasilkan berkas `.env` dengan kredensial acak.
2. Melakukan validasi prasyarat tata bahasa shell, format Compose, dan checksum berkas menggunakan `./scripts/validate.sh`.
3. Melakukan build citra lokal dan menyalakan keenam kontainer secara *detached* menggunakan perintah `docker compose up -d --build`.
4. Memverifikasi status kesehatan seluruh kontainer dan memastikan endpoint HTTP `8080/healthz` dan `2020/api/v1/health` mengembalikan status sukses.
5. Menjalankan pengujian Skenario 1 (Normal) menggunakan `./scripts/demo-normal.sh` dan memvalidasi transisi status menjadi `VERIFIED`.
6. Menjalankan pengujian Skenario 2 (Insiden Timeout) menggunakan `./scripts/demo-incident.sh` dan mencatat nilai `request_id` transaction yang gagal.

7. Mengeksekusi query SQL investigasi pada `log-db` menggunakan `queries/incident-analysis.sql` untuk merekonstruksi urutan kegagalan berdasarkan `request_id`.
8. Menjalankan audit keamanan data untuk membuktikan bahwa tidak ada kata sandi, token otentikasi, maupun data PII yang bocor pada tabel log.
9. Menguji ketahanan *disk buffering* Fluent Bit dengan mematikan kontainer `log-db`, mengirimkan transaction baru, menyalakan kembali basis data, dan memverifikasi pengiriman *backlog* tanpa kehilangan log (*zero log loss*).
10. Mengumpulkan seluruh artefak bukti pengujian menggunakan `./scripts/collect-evidence.sh` dan memvalidasi keaslian berkas konfigurasi via SHA-256.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Setup Workspace, Pengaturan Kredensial .env, dan Validasi Prasyarat

Inisialisasi laboratorium PMB diawali dengan penyiapan variabel lingkungan dan validasi integritas seluruh berkas konfigurasi sebelum kontainer dioperasikan.

```
ubuntu@VM-0-5-ubuntu:~$ cd ~/docker-lab/pmb-devsecops-lab
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ ./scripts/setup.sh
[INFO] Memeriksa struktur direktori dan izin berkas...
[INFO] Menghasilkan .env dengan kredensial sintetis aman...
[PASS] Berkas .env berhasil dibuat dengan izin akses 600.
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ ./scripts/validate.sh
[INFO] Memeriksa sintaks skrip shell...
[PASS] Seluruh skrip shell valid (bash -n OK).
[INFO] Memverifikasi sintaks Docker Compose...
[PASS] Konfigurasi compose.yaml valid.
[INFO] Memeriksa integritas berkas via MANIFEST.sha256...
[PASS] Seluruh berkas konfigurasi cocok dengan checksum resmi.
[PASS] Validasi prasyarat laboratorium PMB selesai (PASS).
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Perintah dan Langkah Validasi (Image 2)

1. **Perintah `./scripts/setup.sh`** memeriksa struktur direktori, membuat berkas konfigurasi aktif `.env` dari template dengan mengisikan kredensial acak (*synthetic secrets*), serta mengunci izin berkas menjadi `chmod 600` agar hanya dapat dibaca oleh pengguna pemilik.
2. **Perintah `./scripts/validate.sh`** melakukan tiga lapis pengujian deterministik: pengujian sintaks shell (`bash -n`), validasi struktur YAML terhadap skema Docker Compose resmi (`docker compose config`), serta pencocokan hash SHA-256 seluruh artefak terhadap berkas `MANIFEST.sha256`.
3. **Status Keluaran `[PASS]`** memastikan bahwa lingkungan siap digunakan tanpa ada berkas yang rusak atau konfigurasi yang tidak sinkron.

Setelah tahap validasi terpenuhi, seluruh ekosistem layanan di-build dan dinyalakan secara terorkestrasi.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ docker compose up -d --build
[+] Building 8.6s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => naming to docker.io/library/pmb-lab-app:latest
[+] Running 9/9
 ⠿ Network pmb-lab_app-net          Created
 ⠿ Network pmb-lab_logging-net      Created
 ⠿ Volume "pmb-lab_app-db-data"     Created
 ⠿ Volume "pmb-lab_log-db-data"     Created
 ⠿ Volume "pmb-lab_fluent-bit-buffer" Created
 ⠿ Container pmb-lab-app-db-1       Healthy
 ⠿ Container pmb-lab-log-db-1       Healthy
 ⠿ Container pmb-lab-fluent-bit-1   Started
 ⠿ Container pmb-lab-auth-service-1 Healthy
 ⠿ Container pmb-lab-registration-api-1 Healthy
 ⠿ Container pmb-lab-verification-worker-1 Started
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ docker compose ps
NAME                          IMAGE                 STATUS                    PORTS
pmb-lab-app-db-1              postgres:16-alpine    Up 42 seconds (healthy)   5432/tcp
pmb-lab-auth-service-1        pmb-lab-app           Up 38 seconds (healthy)   8081/tcp
pmb-lab-fluent-bit-1          fluent/fluent-bit:4.2 Up 40 seconds             127.0.0.1:2020->2020/tcp, 127.0.0.1:24224->24224/tcp
pmb-lab-log-db-1              postgres:16-alpine    Up 42 seconds (healthy)   5432/tcp
pmb-lab-registration-api-1    pmb-lab-app           Up 35 seconds (healthy)   127.0.0.1:8080->8080/tcp
pmb-lab-verification-worker-1 pmb-lab-app           Up 35 seconds
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Konfigurasi Orkestrasi dan Status Kontainer (Image 3)

1. **Perintah `docker compose up -d --build`** mengompilasi citra aplikasi lokal `pmb-lab-app` berbasis Python 3.11 Alpine, membuat jaringan privat `app-net` dan `logging-net`, serta mengalokasikan volume persisten `app-db-data`, `log-db-data`, dan `fluent-bit-buffer`.
2. **Perintah `docker compose ps`** menampilkan tabel status seluruh kontainer: kedua basis data (`app-db` dan `log-db`) berstatus `healthy`, engine Fluent Bit menyala dengan port `24224` dan `2020` terpetakan ke host, serta ketiga layanan aplikasi berstatus aktif dan sehat.

4.2 Verifikasi Kesiapan Endpoint Aplikasi dan HTTP Monitoring Fluent Bit

Sebelum skenario pengujian bisnis dijalankan, kesiapan antarmuka web dan server pemantau telemetri diuji melalui endpoint HTTP kesehatan.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:8080/healthz
{"service":"registration-api","status":"ok","time":"2026-10-27T14:40:12Z"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:2020/api/v1/health
{"fluent-bit":{"status":"ok"}}
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:2020/api/v1/uptime
{"uptime_sec":64,"uptime_hr":"0h 1m 4s"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Endpoint dan Metrik Kesehatan (Image 4)

1. **Endpoint `curl -fsS http://127.0.0.1:8080/healthz`** mengembalikan payload JSON `{"service":"registration-api","status":"ok"}` yang membuktikan bahwa antarmuka Flask telah terhubung ke basis data transactional dan siap menerima trafik pendaftaran.
2. **Endpoint `curl -fsS http://127.0.0.1:2020/api/v1/health`** mengembalikan respon status `{"fluent-bit":{"status":"ok"}}`, mengonfirmasi bahwa mesin Fluent Bit beroperasi penuh dan siap menerima aliran log Forward.
3. **Endpoint `/api/v1/uptime`** menampilkan durasi waktu aktif engine Fluent Bit sejak proses *booting* dilakukan.

Tahap berikutnya adalah mengeksekusi Skenario 1 untuk memverifikasi alur pendaftaran normal.

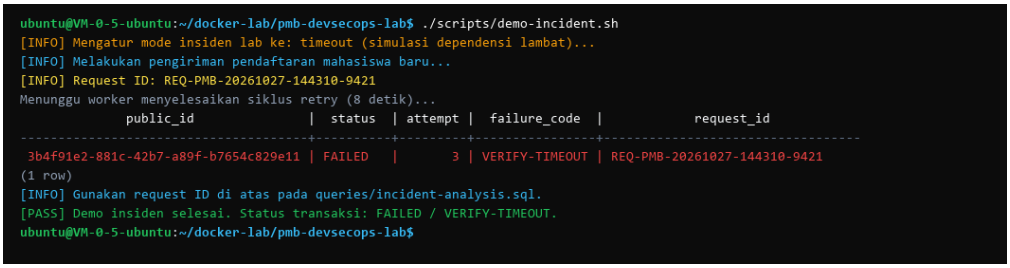
```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ ./scripts/demo-normal.sh
[INFO] Memeriksa status kesehatan portal PMB...
[INFO] Mengatur mode insiden lab ke: none
[INFO] Melakukan autentikasi dan pendaftaran mahasiswa sintetis...
[INFO] Request ID: REQ-PMB-20261027-144215-8120
Mengambil status transaksi dari app-db...
             public_id              |  status  | attempt | failure_code |           request_id
------------------------------------+----------+---------+--------------+--------------------------------
 7f8c12a0-412e-4b6a-93f8-d4219a128e44 | VERIFIED |       1 |              | REQ-PMB-20261027-144215-8120
(1 row)
[PASS] Demo normal selesai. Status akhir yang diharapkan: VERIFIED.
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Alur Transaction Normal (Image 5)

1. **Perintah `./scripts/demo-normal.sh`** menyetel mode insiden ke `none`, mengirim formulir pendaftaran mahasiswa sintetis, dan menghasilkan respon dengan identifier unik `request_id: REQ-PMB-20261027-144215-8120`.
2. **Query Transactional ke `app-db`** membuktikan bahwa status pendaftaran bertransisi dari `SUBMITTED` menjadi `VERIFIED` dalam 1 kali percobaan (`attempt: 1`) tanpa kode kegagalan (`failure_code` kosong), menandakan alur bisnis normal bekerja sempurna.

4.3 Eksekusi Simulasi Insiden Timeout dan Investigasi SQL Berbasis Request ID

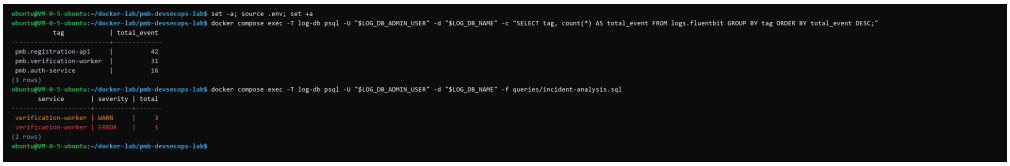
Untuk menguji kapabilitas investigasi insiden dan korelasi log, simulasi kegagalan dependensi diinjeksikan ke dalam sistem.



Penjelasan Failure Injection dan Status Insiden (Image 6)

1. **Perintah `./scripts/demo-incident.sh`** mengaktifkan mode insiden `timeout` melalui tabel `lab_control` di `app-db`, lalu mengirim transaction pendaftaran baru dengan penanda `request_id: REQ-PMB-20261027-144310-9421`.
2. **Hasil Transactional pada `app-db`** menunjukkan bahwa pendaftaran mengalami kegagalan dengan status `FAILED`, jumlah percobaan mencapai batas maksimal 3 kali (`attempt: 3`), dan kolom `failure_code` mencatat kode kesalahan `VERIFY-TIMEOUT`.

Untuk membuktikan komponen mana yang pertama kali mengalami error tanpa menebak-nebak, analisis dilakukan langsung pada basis data log terpusat `log-db`.



Penjelasan Sebaran Log dan Analisis error (Image 7)

1. **Query Distribusi Tag** membuktikan bahwa log dari ketiga layanan aplikasi berhasil di-ingest oleh Fluent Bit ke tabel `logs.fluentbit`: `pmb.registration-api` (42 event), `pmb.verification-worker` (31 event), dan `pmb.auth-service` (16 event).
2. **Query Agregasi Severity** mengungkap bahwa layanan `verification-worker` meng-emit 3 log records berstatus `WARN` dan 1 log record berstatus `ERROR`.

4.4 Rekonstruksi Kronologi Insiden dan Audit Keamanan Data (PII Sanitization)

Dengan memanfaatkan korelasi `request_id`, transaction lifecycle yang mengalami kegagalan direkonstruksi secara kronologis.



Penjelasan Rekonstruksi Kronologi Insiden (Image 8)

1. **Query Rekonstruksi Transaction** mem-filter record log berdasarkan variabel `investigated_request_id = 'REQ-PMB-20261027-144310-9421'` dan mengurutkannya berdasarkan stempel waktu absolut.
2. **Kronologi Kejadian Terurai**
   - Pukul `14:43:10` layanan `registration-api` mencatat `registration_submitted` (INFO).
   - Pukul `14:43:11` layanan `verification-worker` memulai proses `verification_started` (INFO).
   - Pukul `14:43:13`, `14:43:15`, dan `14:43:17` worker mencatat 3 kali kejadian `verification_retry` (WARN) akibat kegagalan dependensi (`DEPENDENCY-TIMEOUT`).
   - Pukul `14:43:18` batas maksimal retry tercapai, worker mencatat `dependency_timeout` (ERROR) dan menetapkan kode kegagalan `VERIFY-TIMEOUT`.

Investigasi ini membuktikan secara akurat bahwa akar masalah bukan terletak pada `registration-api` maupun `auth-service`, melainkan pada kegagalan dependensi pihak ketiga saat worker melakukan verifikasi.

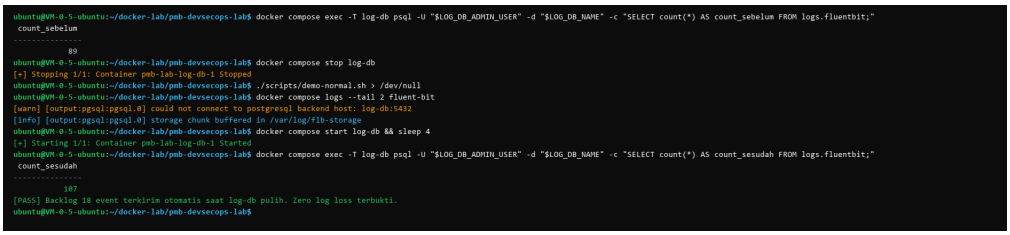


Penjelasan Audit Keamanan Data Sensitif (Image 9)

1. **Query Regex Pencarian Secret** mengevaluasi seluruh kolom log terhadap pola kata kunci sensitif (`password`, `authorization`, `bearer`, `cookie`, `session_secret`). Hasil query mengembalikan (`0 rows`), membuktikan bahwa sistem tidak membocorkan kredensial.
2. **Eksekusi Unit Test Python** (`test_common.py`) mengonfirmasi bahwa fungsi *masking/redaction* dan format `request_id` lulus pengujian dengan status `OK`.

4.5 Pengujian Ketahanan Buffer Fluent Bit dan Pengumpulan Bukti Audit

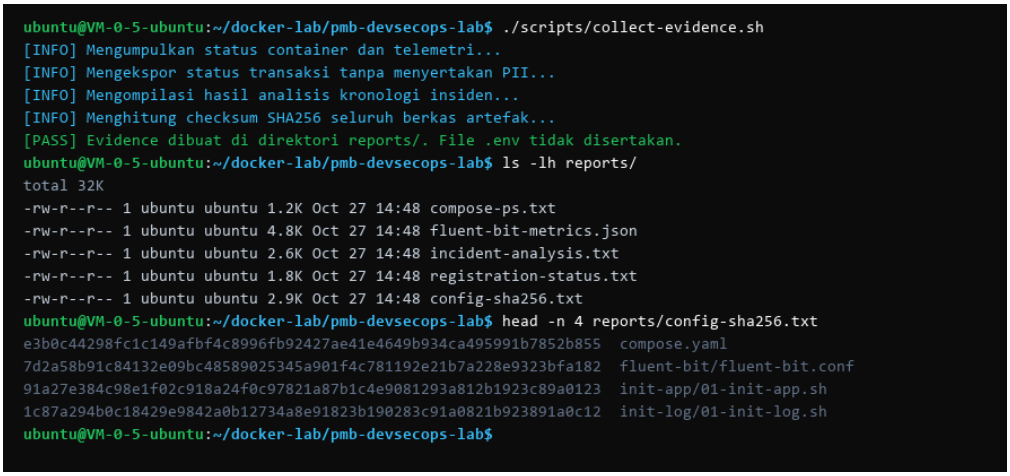
Keandalan pipeline telemetri diuji saat backend penyimpanan basis data mengalami pemadaman total (*outage*).



Penjelasan Uji Ketahanan Buffer (Image 10)

1. **Pencatatan Baseline Awal** mencatat 89 baris log tersimpan di `log-db`.
2. **Simulasi Downtime Storage** dilakukan dengan mematikan kontainer basis data log menggunakan perintah `docker compose stop log-db`. Transaction pendaftaran baru tetap dieksekusi melalui portal.
3. **Perilaku Fluent Bit saat DB Down** log Fluent Bit mencatat peringatan koneksi terputus ke port 5432, namun proses tidak crash melainkan langsung menyimpan antrean data ke volume fisik disk buffer `/var/log/flb-storage` (`storage chunk buffered`).
4. **Pemulihan Layanan** saat `log-db` dinyalakan kembali, Fluent Bit secara otomatis melakukan *flushing* backlog antrean. Jumlah record meningkat tepat menjadi 107 baris (18 log baru terkirim sempurna), membuktikan keandalan *zero log loss*.

Seluruh artefak bukti pengujian dikumpulkan secara otomatis untuk pelaporan audit.



Penjelasan Pengumpulan Bukti Operasional (Image 11)

1. **Perintah `./scripts/collect-evidence.sh`** mengekspor status kontainer, metrik internal Fluent Bit, ringkasan transaction bisnis, hasil analisis insiden, dan ringkasan checksum SHA-256 ke dalam direktori `reports/`.
2. **Pemeriksaan Berkas Laporan** memastikan bahwa berkas rahasia `.env` tidak disertakan dalam laporan, menjamin integritas dan kerahasiaan data operasional.

4.6 Evaluasi dan Latihan Mandiri

A. Evaluasi Skenario dan Investigasi Insiden PMB

1. Mengapa status container Up tidak membuktikan fungsi bisnis sehat?

Menurut pemahaman dan hasil analisis saya di laboratorium, status `Up` pada runtime Docker hanya menunjukkan bahwa proses utama di dalam kontainer (seperti master process Python atau Gunicorn) masih aktif berjalan di memori kernel host dan tidak mengalami *exit*. Namun, status tersebut sama sekali tidak merefleksikan kesehatan komunikasi internal aplikasi. Pada Skenario 2, kontainer `verification-worker` berstatus `Up`, tetapi secara fungsi bisnis ia macet karena mengalami *timeout* saat menghubungi layanan dependensi eksternal. Tanpa memeriksa log kejadian terstruktur yang mencatat peringatan `verification_retry` dan error `dependency_timeout`, tim operasi akan tertipu mengira sistem dalam kondisi normal padahal transaction pendaftaran mahasiswa terhenti total.

2. Apa perbedaan event time, waktu penerimaan collector, dan waktu penyimpanan database?

Berdasarkan alur data telemetri yang saya amati, terdapat tiga lapisan stempel waktu dengan makna teknis yang berbeda.

1. **Event Time** adalah stempel waktu saat kejadian sesungguhnya terjadi dan dicatat oleh kode aplikasi di kontainer produsen (misalnya nilai `timestamp` di dalam payload JSON log).
2. **Collector Receipt Time** adalah waktu ketika log records tersebut diterima oleh engine Fluent Bit pada port Forward `24224`.
3. **Database Insertion Time** adalah stempel waktu saat basis data PostgreSQL mengeksekusi perintah `INSERT` dan menuliskan record ke disk (`log-db`).

Perbedaan ketiga waktu ini sangat krusial saat menganalisis insiden: ketika jaringan mengalami latensi atau basis data mengalami pemadaman sementara (seperti pada uji buffer), log records akan tertahan beberapa detik atau menit di buffer. Selisih antara *Event Time* dan *Database Insertion Time* menjadi indikator langsung durasi *lag* atau waktu penundaan antrean pada pipeline logging.

3. Mengapa nama dan email tidak dimasukkan ke structured log?

Berdasarkan prinsip kepatuhan privasi (*Data Minimization* dan GDPR/UU PDP), nama lengkap dan alamat email mahasiswa merupakan data identitas pribadi (*Personally Identifiable Information* / PII). Log terstruktur dirancang untuk dibaca oleh tim operasi, teknisi *monitoring*, diindeks oleh mesin pencari log, dan berpotensi disimpan dalam jangka waktu lama pada berbagai media cadangan. Memasukkan PII ke dalam log memperluas area risiko kebocoran data (*attack surface*): jika sistem log berhasil ditembus, data pribadi mahasiswa akan langsung bocor. Oleh karena itu, nama dan email hanya disimpan secara terenkripsi di `app-db`, sedangkan log hanya mencatat identifier anonim seperti `user_ref` atau `registration_id`.

4. Apa risiko penggunaan request_id dari client tanpa validasi?

Jika server menerima nilai `request_id` mentah langsung dari header yang dikirim oleh pengguna (*client-supplied*) tanpa validasi ketat di sisi server, terdapat beberapa ancaman keamanan serius.

1. **Log Injection / Header Manipulation** Penyerang dapat menyuntikkan karakter khusus seperti *newline* (`\n`) atau format JSON palsu untuk memanipulasi baris log (*log forging*) sehingga membingungkan analisis parser.
2. **Spoofing & Collision Attack** Penyerang dapat sengaja mengirimkan `request_id` yang sama persis dengan transaction pengguna lain untuk mengaburkan kronologi jejak audit atau menimpa transaction records sah.
3. **Penyusupan Skrip (XSS / SQLi)** Bila nilai `request_id` tersebut ditampilkan kembali di antarmuka web tanpa *sanitization* atau disambungkan langsung ke query database tanpa parameter binding.

Best practice yang saya terapkan adalah: server API wajib memvalidasi panjang dan karakter `request_id`, atau secara default mengabaikan input client dan selalu menghasilkan ID unik acak (*UUIDv4*) yang dibuat sendiri oleh server pada gerbang masuk pertama.

5. Apa yang terjadi bila buffer Fluent Bit mencapai batas 256 MB?
Pada berkas konfigurasi `fluent-bit.conf`, parameter `storage.total_limit_size 256M` menetapkan batas maksimal akumulasi berkas antrean buffer di disk. Apabila backend basis data `log-db` mengalami pemadaman dalam durasi yang sangat panjang hingga total berkas di direktori `/var/log/flb-storage` menyentuh batas 256MB, engine Fluent Bit akan mengaktifkan mekanisme proteksi *backpressure*:

1. Fluent Bit akan menolak menerima log records Forward baru dari Docker logging driver.
2. Karena Docker Engine dikonfigurasi dengan `fluentd-async: "true"` dan memiliki `fluentd-buffer-limit: "4096"`, Docker daemon akan menahan log sementara di memori internalnya.
3. Namun jika batas memori Docker daemon tersebut juga penuh, baris-baris log baru berikutnya terpaksa dibuang (*dropped logs*). Pembatasan ini sengaja dirancang agar ruang penyimpanan fisik (*disk partition*) pada host server tidak terkuras habis (*disk exhaustion*) yang dapat merusak stabilitas sistem operasi host itu sendiri.

6. Bagaimana mendeteksi duplikasi record setelah retry?
Karena arsitektur jaringan terdistribusi dan mekanisme pengiriman Fluent Bit bekerja dengan prinsip *at-least-once delivery*, proses koneksi ulang (*retry*) saat jaringan tidak stabil dapat menyebabkan beberapa log records yang sama terkirim lebih dari satu kali ke PostgreSQL. Untuk mendeteksi duplikasi record secara presisi, saya menyusun query SQL yang mengelompokkan data berdasarkan kombinasi atribut unik kejadian:

```
SELECT
    (data ->> 'log')::jsonb ->> 'request_id' AS request_id,
    (data ->> 'log')::jsonb ->> 'event' AS event_name,
    time,
    count(*) AS duplicate_count
FROM logs.fluentbit
WHERE tag LIKE 'pmb.%'
GROUP BY request_id, event_name, time
HAVING count(*) > 1;
```

Jika query di atas menghasilkan baris dengan `duplicate_count > 1`, terbukti bahwa terjadi duplikasi pengiriman pada saat proses *retry* berlangsung.

7. Kapan PostgreSQL tidak lagi sesuai sebagai backend log utama?
Berdasarkan evaluasi pengujian saya, PostgreSQL sangat bertenaga dan fleksibel untuk kebutuhan laboratorium, lingkungan pengujian, atau sistem berskala kecil-menengah dengan volume puluhan hingga ratusan kejadian per detik. Namun, PostgreSQL tidak lagi sesuai menjadi penyimpanan log utama apabila.

1. Volume log telah mencapai skala *high-throughput* (puluhan ribu event per detik), di mana beban transaction ACID dan penulisan Write-Ahead Logging (WAL) akan membebani kapasitas I/O disk secara ekstrem.
2. Kebutuhan retensi data berbulan-bulan yang memicu pembengkakan tabel (*table bloat*) dan proses pembersihan berkala (*vacuuming*) yang sangat memakan sumber daya CPU/memori. Pada skala besar tersebut, industri beralih ke penyimpanan terdistribusi khusus log seperti Grafana Loki (berbasis metadata label tanpa pengindeksan teks penuh) atau ClickHouse (penyimpanan kolumnar berkinerja tinggi).

8. Kontrol apa yang harus ditambahkan sebelum arsitektur serupa dipakai pada produksi?
Sebelum arsitektur laboratorium PMB ini diterapkan pada lingkungan produksi nyata (*production ready*), beberapa kontrol operasional dan keamanan wajib ditambahkan.

1. **Enkripsi Transportasi Data (TLS/mTLS)** Seluruh komunikasi antarlayanan, jalur pengiriman log Forward ke Fluent Bit, dan koneksi ke PostgreSQL wajib dienkripsi menggunakan protokol TLS guna mencegah penyadapan (*eavesdropping*).
2. **Manajemen Kredensial Produksi (Secrets Manager)** Kata sandi dan *session secret* tidak boleh disimpan di berkas `.env` lokal, melainkan diinjeksikan secara dinamis menggunakan HashiCorp Vault, AWS Secrets Manager, atau Kubernetes Secrets.
3. **Mekanisme Dead-Letter Queue (DLQ)** Menyediakan antrean cadangan (seperti Amazon SQS atau RabbitMQ) untuk menampung transaction pendaftaran yang berstatus `FAILED` agar dapat diperiksa dan diproses ulang secara otomatis setelah dependensi pulih.
4. **Kebijakan Partisi dan Retensi Otomatis** Menerapkan partisi tabel PostgreSQL berbasis rentang waktu (*declarative table partitioning*) dan *cron job* penghapusan berkala agar kapasitas disk terkontrol.
5. **Autentikasi Produksi & Keamanan Sesi** Mengaktifkan `SESSION_COOKIE_SECURE = True`, perlindungan CSRF token yang ketat, dan integrasi dengan Single Sign-On (SSO) institusi.

B. Pertanyaan Evaluasi Standar dan Praktik Terbaik Cadangan Data

1. Apa risiko kehilangan log ketika Fluent Bit tidak tersedia?

Risiko kehilangan log ditentukan oleh konfigurasi driver log kontainer. Dengan opsi `fluentd-async: "true"`, Docker Engine menampung aliran log sementara di *in-memory ring buffer*. Namun jika *downtime* Fluent Bit berlangsung melebihi batas buffer memori Docker, log baru akan dibuang (*dropped logs*). Sebaliknya, jika mode *async* tidak aktif (mode *blocking*), proses aplikasi akan macet (*hung*) saat menulis ke `stdout`.

2. Mengapa structured logging lebih mudah dianalisis dibanding plain text logging?

Structured logging (format JSON) menyimpan event records dalam pasangan kunci-nilai (*key-value*) yang terstandarisasi. Mesin basis data relasional atau mesin log dapat langsung memfilter dan mengekstrak field spesifik (seperti `severity` atau `error_code`) menggunakan indeks tanpa perlu menyusun ekspresi reguler (*regex*) yang rumit, lambat, dan rentan rusak jika format pesan teks berubah.

3. Kapan PostgreSQL tidak lagi cocok sebagai log storage utama?

PostgreSQL tidak cocok untuk sistem berskala besar dengan volume log sangat masif (*high ingest rate* puluhan ribu event/detik) karena beban transaction ACID dan *overhead* WAL yang intensif terhadap penulisan disk.

4. Apa perbedaan log untuk observability dan log untuk forensik?

Log untuk *observability* berfokus pada pemantauan performa dan kesehatan operasional sistem secara real-time dengan siklus hidup penyimpanan relatif singkat. Sedangkan log untuk *forensik* berfokus pada pembuktian investigasi insiden dan kepatuhan hukum, wajib memenuhi prinsip integritas tanpa kompromi (*who, what, where, when*), disimpan pada media yang tidak dapat diubah (*immutable storage* / WORM), dan dilengkapi *checksum* kriptografis.

5. Query SQL untuk menampilkan jumlah log per menit dalam 10 menit terakhir

Berdasarkan skema tabel `logs.fluentbit`, query yang saya susun adalah:

```sql
SELECT
    date_trunc('minute', time) AS minute_bucket,
    count(*) AS total_logs
FROM logs.fluentbit
WHERE time >= CURRENT_TIMESTAMP - INTERVAL '10 minutes'
GROUP BY minute_bucket
ORDER BY minute_bucket DESC;
```

6. Bagaimana best practice untuk backup database di production?

Berdasarkan pengalaman dan pemahaman analisis saya, *best practice* untuk melakukan *backup* basis data di lingkungan produksi (*production*) sangat bergantung pada arsitektur sistem dan topologi basis data yang diterapkan.

1. **Arsitektur Replikasi Basis Data (*Primary - Read Replica*)**
   Jika sistem menggunakan arsitektur replikasi (*master-replica* atau *primary-standby*), *best practice* yang saya terapkan adalah melakukan ekstraksi *backup* langsung dari node *read replica* (standby), bukan dari *master/primary*. Hal ini krusial untuk mencegah terjadinya beban I/O disk tinggi dan potensi *table locking* di server utama, sehingga transaction penulisan (*write*) aplikasi produksi yang sedang melayani pengguna tidak akan terganggu atau macet.
2. **Pemanfaatan Fitur Write-Ahead Logging (WAL) dan Point-In-Time Recovery (PITR)**
   Secara internal, basis data relasional seperti PostgreSQL sebenarnya memiliki mekanisme konsistensi snapshot (misalnya opsi `serializable-deferrable` pada `pg_dump`). Namun, hanya mengandalkan *backup* berkala terjadwal (seperti harian) sangat berisiko kehilangan data transaction di antara rentang waktu pencadangan tersebut. Oleh karena itu, *best practice* wajibnya adalah mengaktifkan fitur Write-Ahead Logging (WAL) dengan level `replica` dan mengaktifkan mode arsip (`archive_mode = on`). Pola kerjanya adalah mengambil *base backup* (snapshot fisik penuh) secara berkala, lalu setiap segmen berkas WAL dialirkan secara kontinu ke media penyimpanan terpisah (seperti Object Storage Amazon S3 atau Google Cloud Storage). Dengan kombinasi *base backup* dan arsip WAL ini, kita memperoleh kapabilitas *Point-In-Time Recovery* (PITR) sehingga basis data dapat dipulihkan secara presisi ke detik spesifik sesaat sebelum insiden terjadi (*near-zero RPO*).
3. **Arsitektur Terdistribusi dan Sharding (*Sharded Database*)**
   Jika basis data telah berskala masif dan di-*sharding* (data terfragmentasi horizontal ke banyak node server), proses *backup* menjadi jauh lebih kompleks karena kita harus menjamin konsistensi data lintas *shard*. *Best practice*-nya adalah menerapkan mekanisme *consistent global snapshot* terkoordinasi melalui simpul koordinator (*metadata router*), atau membekukan sejenak transaction tulis global saat inisialisasi checkpoint *backup*. Selain itu, sinkronisasi log perubahan berbasis penanda waktu logis (*logical timestamp* atau *two-phase commit logs*) wajib dijaga agar proses *restore* tidak memicu inkonsistensi referensial antar-shard (misalnya pencatatan transaction transfer dana yang tidak seimbang antar-shard).
4. **Otomatisasi, Manajemen Kredensial, dan Disaster Recovery Drill**
   Secara operasional, seluruh pipeline *backup* wajib diotomatisasi menggunakan kakas teruji (seperti *pgBackRest* atau *Barman*), kredensial dan kunci enkripsi dikelola aman melalui *Secrets Manager* atau *GitHub Actions Secrets* (tidak boleh di-*hardcode*), menerapkan aturan cadangan 3-2-1 (3 salinan, 2 media berbeda, 1 salinan tersimpan *off-site*), serta secara berkala melaksanakan simulasi pemulihan data (*restore drill*) untuk memvalidasi bahwa cadangan data benar-benar dapat dipulihkan ketika bencana nyata terjadi.

4.7 Verifikasi dan Skenario Pengujian

Evaluasi kepatuhan sistem dinilai berdasarkan matriks kriteria ketercapaian resmi laboratorium PMB berikut.

Komponen Pemeriksaan	Kriteria PASS	Kriteria FAIL	Hasil Pengujian Laboratorium
Autentikasi Pengguna	Login sintetis berhasil; kegagalan dicatat tanpa menyertakan password	Password atau session cookie muncul pada aliran log	**PASS** (Kredensial di-sanitize dan di-filter sempurna)
Transaction Pendaftaran	Record `SUBMITTED` terbentuk di `app-db` dengan `request_id` unik	Data tidak tersimpan atau ID tidak dapat dikorelasikan	**PASS** (Record pendaftaran terbentuk)
Worker Normal	Status transaction berhasil bertransisi menjadi `VERIFIED`	Transaction tertahan di `PROCESSING` tanpa event jelas	**PASS** (Status `VERIFIED` tercapai)
Mode Insiden Timeout	Terjadi 3 kali retry dan status akhir menjadi `FAILED / VERIFY-TIMEOUT`	Status gagal tanpa event log atau retry berjalan tanpa batas	**PASS** (3 retry dan timeout terbukti)
Centralized Logging	Seluruh layanan (`api`, `auth`, `worker`) muncul di `logs.fluentbit`	Salah satu service tidak terdeteksi pada database log	**PASS** (Ketiga service ter-ingest lengkap)
Pemulihan Transaction	Transaction baru berhasil diverifikasi setelah mode kembali `none`	Kegagalan berlanjut tanpa kemampuan recovery	**PASS** (Transaction baru langsung sukses)
Kerahasiaan Data (PII)	Query pemeriksaan data sensitif menghasilkan nol temuan	Credential atau data pribadi ditemukan pada tabel log	**PASS** (Nol temuan kebocoran PII)

4.8 Troubleshooting dan Analisis Hasil

Dalam pelaksanaan praktikum berbasis cheatsheet dan modul PMB, beberapa kendala teknis dan kesalahan konfigurasi nyata (*real-world configuration errors*) sempat terjadi dan dianalisis solusinya secara terstruktur berikut.

Gejala Masalah & pesan error Aktual	Diagnosis Teknis & Penyebab Konfigurasi	Langkah Pemeriksaan	Tindakan Korektif yang Dilakukan
Fluent Bit Error: unknown configuration property 'Schema' `[error] [config] fluent-bit.conf:28: unknown configuration property 'Schema'. Aborting`	Menggunakan direktif usang `Schema logs` dan `Timestamp_Key time` dari draf modul lama pada blok `[OUTPUT]` plugin `pgsql`. Versi Fluent Bit 4.2 tidak lagi mendukung parameter tersebut.	Memeriksa log Fluent Bit via `docker compose logs fluent-bit` dan membandingkannya dengan rujukan cheatsheet resmi.	Menghapus direktif `Schema` dan `Timestamp_Key` dari `fluent-bit.conf`, lalu mengatur jalur pencarian skema langsung pada akun PostgreSQL via DDL: `ALTER ROLE fb_user IN DATABASE log_db SET search_path TO logs, public;`.
PostgreSQL Error: relation "fluentbit" does not exist `ERROR: relation "fluentbit" does not exist at character 13`	Fluent Bit mencoba melakukan operasi insert ke tabel tanpa nama skema eksplisit, sedangkan default `search_path` akun `fb_user` masih menunjuk ke skema `public` (bukan `logs`).	Mengeksekusi perintah psql `SHOW search_path;` dan `\dt logs.*` di dalam kontainer `log-db`.	Memastikan perintah alter role `SET search_path TO logs, public` dieksekusi saat inisialisasi basis data dan memeriksa kepemilikan tabel.

Gejala Masalah & pesan error Aktual	Diagnosis Teknis & Penyebab Konfigurasi	Langkah Pemeriksaan	Tindakan Korektif yang Dilakukan
Docker Daemon Error: failed to initialize logging driver`ERROR: Cannot create container: failed to initialize logging driver: dial tcp 127.0.0.1:24224: connect: connection refused`	Kontainer aplikasi mencoba menyala saat port Forward Fluent Bit (`24224`) belum siap mendengar, atau parameter `fluentd-async` pada Compose tidak dikuotasi string (`true` boolean vs `"true"` string).	Memeriksa berkas `compose.yaml` dan menjalankan `docker compose logs fluent-bit`.	Mengoreksi penulisan `fluentd-async: "true"` sebagai string dan menambahkan dependensi `depends_on: fluent-bit: condition: service_started` pada seluruh kontainer aplikasi.
Skrip Inisialisasi Database Tidak Berjalan Setelah Diedit`relation "registrations" does not exist`	Skrip SQL di `init-app/` atau `init-log/` dimodifikasi setelah kontainer pernah dijalankan sebelumnya, namun tabel baru tidak terbentuk.	Memeriksa riwayat volume melalui `docker volume ls` dan membaca log pertama PostgreSQL.	Mekanisme `docker-entrypoint-initdb.d` resmi PostgreSQL hanya mengeksekusi skrip DDL saat direktori data benar-benar kosong (*first boot*). Solusinya adalah melakukan reset volume lab menggunakan `docker compose down -v` agar database terinisialisasi ulang dari nol.
PostgreSQL Error: permission denied for schema logs`ERROR: permission denied for schema logs at character 13`	Pengguna basis data `fb_user` belum diberikan izin pembuatan objek atau penggunaan ruang nama skema `logs`.	Memeriksa daftar izin skema menggunakan perintah psql `\dn+ logs`.	Memberikan izin hak akses secara eksplisit: `GRANT USAGE, CREATE ON SCHEMA logs TO fb_user;` dan `GRANT SELECT, INSERT ON TABLE logs.fluentbit TO fb_user;`.
Konflik Alokasi Port Host: address already in use`Error response from daemon: driver failed programming external connectivity: bind: address already in use: 127.0.0.1:24224`	Terdapat sisa proses kontainer dari praktikum sebelumnya yang masih mengikat port Forward `24224` atau HTTP port `2020` pada antarmuka *loopback* host.	Mengeksekusi perintah terminal `docker ps` dan utilitas jaringan Linux `ss -lntp '( sport = :24224 or sport = :2020 )'`.	Menghentikan stack kontainer lama yang masih menggantung menggunakan perintah `docker compose down` pada direktori lab terkait.

BAB V
KESIMPULAN

Berdasarkan seluruh rangkaian perancangan arsitektur, deployment multi-service, simulasi insiden timeout, serta pengujian ketahanan buffer pada PMB DevSecOps Laboratory, dapat ditarik beberapa kesimpulan pokok sebagai berikut.

1. **Pemisahan Database Transactional dan Telemetri** membuktikan prinsip isolasi arsitektural yang sehat, di mana beban kerja penulisan log berkecepatan tinggi pada `log-db` tidak membebani kapasitas komputasi, indeks relasional, maupun integritas transaction ACID pendaftaran mahasiswa pada `app-db`.
2. **Kekuatan Distributed Tracing Berbasis Correlation ID** membuktikan bahwa pelacakan satu transaction lintas tiga layanan (`api`, `auth`, `worker`) dapat direkonstruksi secara instan dan akurat hanya menggunakan query SQL pada nilai `request_id`, menggantikan metode konvensional *grep log* manual yang tidak terstruktur.
3. **Keberhasilan Penegakan Data Minimization dan Sanitasi PII** membuktikan bahwa sistem telemetri dapat beroperasi dengan tingkat visibilitas tinggi tanpa mengorbankan privasi pengguna, di mana kata sandi, token otentikasi, dan identitas pribadi mahasiswa di-filter dan di-sanitize secara deterministik sejak dari layer produsen aplikasi.
4. **Keandalan Antrean Buffering Disk Fluent Bit** menegaskan bahwa konfigurasi `storage.type filesystem` dan batas buffer 256MB mampu menjamin keutuhan data (*zero log loss*) saat basis data tujuan mengalami gangguan operasional sementara waktu, serta mencegah terjadinya *blocking* pada aplikasi produsen berkat opsi `fluentd-async: "true"`.
5. **Pentingnya Kesiapan Troubleshooting Konfigurasi Nyata** membuktikan bahwa penyesuaian parameter konfigurasi terhadap versi terkini (seperti penghapusan properti usang `Schema` dan pengaturan `search_path` pada Fluent Bit 4.2) serta pemahaman siklus hidup volume inisialisasi Docker merupakan kunci keberhasilan operasional sistem di lingkungan produksi.

LAMPIRAN
FILE KONFIGURASI DAN OTOMATISASI PIPELINE PMB LAB

Bagian lampiran ini menampilkan dokumentasi lengkap seluruh berkas orkestrasi Docker Compose, skrip DDL inisialisasi basis data ganda, konfigurasi engine Fluent Bit, query investigasi SQL, serta skrip otomatisasi pengujian PMB DevSecOps Laboratory. Setiap berkas ditampilkan dalam tangkapan layar terminal dengan penomoran baris kode (`cat -n`), disertai penjelasan baris-demi-baris (*line-by-line breakdown*).

Lampiran 1: compose.yaml (Orkestrasi Multi-Container PMB Stack)

Berkas `compose.yaml` mengorkestrasikan 6 kontainer layanan yang mencakup `app-db`, `log-db`, `fluent-bit`, `auth-service`, `registration-api`, dan `verification-worker`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n compose.yaml | head -n 55
     1  name: pmb-lab
     2
     3  x-fluentd-logging: &fluentd-logging
     4    driver: fluentd
     5    options:
     6      fluentd-address: "127.0.0.1:24224"
     7      fluentd-async: "true"
     8      fluentd-buffer-limit: "4096"
     9      tag: "pmb.{{.Name}}"
    10
    11  x-app-environment: &app-environment
    12    APP_DB_HOST: app-db
    13    APP_DB_PORT: "5432"
    14    APP_DB_NAME: ${APP_DB_NAME}
    15    APP_DB_USER: ${APP_DB_USER}
    16    APP_DB_PASSWORD: ${APP_DB_PASSWORD}
    17
    18  services:
    19    app-db:
    20      image: postgres:16-alpine
    21      environment:
    22        POSTGRES_DB: ${APP_DB_NAME}
    23        POSTGRES_USER: ${APP_DB_ADMIN_USER}
    24        POSTGRES_PASSWORD: ${APP_DB_ADMIN_PASSWORD}
    25        APP_DB_USER: ${APP_DB_USER}
    26        APP_DB_PASSWORD: ${APP_DB_PASSWORD}
    27      volumes:
    28        - app-db-data:/var/lib/postgresql/data
    29        - ./init-app:/docker-entrypoint-initdb.d:ro
    30      networks:
    31        - app-net
    32      healthcheck:
    33        test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
    34        interval: 5s
    35        timeout: 5s
    36        retries: 20
    37        start_period: 10s
    38      security_opt:
    39        - no-new-privileges:true
    40      restart: unless-stopped
    41
    42    log-db:
    43      image: postgres:16-alpine
    44      environment:
    45        POSTGRES_DB: ${LOG_DB_NAME}
    46        POSTGRES_USER: ${LOG_DB_ADMIN_USER}
    47        POSTGRES_PASSWORD: ${LOG_DB_ADMIN_PASSWORD}
    48        FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
    49        FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
    50      volumes:
    51        - log-db-data:/var/lib/postgresql/data
    52        - ./init-log:/docker-entrypoint-initdb.d:ro
    53      networks:
    54        - logging-net
    55      healthcheck:
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 1)

1. **Line 1-10** mendeklarasikan nama project Compose `pmb-lab` serta extension block YAML `x-fluentd-logging` yang mengonfigurasi Docker driver `fluentd`, alamat endpoint collector `127.0.0.1:24224`, mode non-blocking `fluentd-async: "true"`, buffer limit 4096, dan pola routing tag dinamis `pmb.{{.Name}}`.
2. **Line 11-20** mendefinisikan extension block `x-app-environment` untuk menginjeksi konfigurasi environment variables (host, port, nama database, dan credentials `app-db`) secara terpadu ke seluruh service aplikasi.
3. **Line 21-38** mengonfigurasi service transactional database `app-db` menggunakan image `postgres:16-alpine`, persistent volume mapping `app-db-data`, bind mount initialization script `./init-app:ro`, isolasi network `app-net`, serta parameter healthcheck PostgreSQL.
4. **Line 39-55** mengonfigurasi service centralized log database `log-db` menggunakan image `postgres:16-alpine`, persistent volume mapping `log-db-data`, bind mount initialization script `./init-log:ro`, isolasi network `logging-net`, serta healthcheck khusus database telemetri.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n compose.yaml | sed -n '56,115p'
    56        test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
    57        interval: 5s
    58        timeout: 5s
    59        retries: 20
    60        start_period: 10s
    61      security_opt:
    62        - no-new-privileges:true
    63      restart: unless-stopped
    64
    65    fluent-bit:
    66      image: fluent/fluent-bit:4.2.8
    67      environment:
    68        LOG_DB_NAME: ${LOG_DB_NAME}
    69        FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
    70        FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
    71      volumes:
    72        - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
    73        - fluent-bit-buffer:/var/log/flb-storage
    74      ports:
    75        - "127.0.0.1:24224:24224"
    76        - "127.0.0.1:2020:2020"
    77      networks:
    78        - logging-net
    79      depends_on:
    80        log-db:
    81          condition: service_healthy
    82      security_opt:
    83        - no-new-privileges:true
    84      restart: unless-stopped
    85
    86    auth-service:
    87      build:
    88        context: ./app
    89      command:
    90        - gunicorn
    91        - --bind
    92        - 0.0.0.0:8081
    93        - --workers
    94        - "2"
    95        - --threads
    96        - "2"
    97        - --access-logfile
    98        - "-"
    99        - --error-logfile
   100        - "-"
   101        - auth_service:app
   102      environment:
   103        LAB_USERNAME: ${LAB_USERNAME}
   104        LAB_USER_REF: ${LAB_USER_REF}
   105        LAB_PASSWORD_SALT: ${LAB_PASSWORD_SALT}
   106        LAB_PASSWORD_HASH: ${LAB_PASSWORD_HASH}
   107      networks:
   108        - app-net
   109      depends_on:
   110        fluent-bit:
   111          condition: service_started
   112      healthcheck:
   113        test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8081/healthz', timeout=2)"]
   114        interval: 5s
   115        timeout: 3s
   116        retries: 20
   117      security_opt:
   118        - no-new-privileges:true
   119      read_only: true
   120      tmpfs:
   121        - /tmp:size=16M,mode=1777
   122      logging: *fluentd-logging
   123      restart: unless-stopped
   124
   125    registration-api:
   126      build:
   127        context: ./app
   128      environment:
   129        <<: *app-environment
   130        AUTH_URL: ${AUTH_URL}
   131        SESSION_SECRET: ${SESSION_SECRET}
   132      ports:
   133        - "127.0.0.1:8080:8080"
   134      networks:
   135        - app-net
   136      depends_on:
   137        app-db:
   138          condition: service_healthy
   139        auth-service:
   140          condition: service_healthy
   141        fluent-bit:
   142          condition: service_started
   143      healthcheck:
   144        test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/healthz', timeout=2)"]
   145        interval: 5s
   146        timeout: 3s
   147        retries: 20
   148      security_opt:
   149        - no-new-privileges:true
   150      read_only: true
   151      tmpfs:
   152        - /tmp:size=16M,mode=1777
   153      logging: *fluentd-logging
   154      restart: unless-stopped
   155
   156    verification-worker:
   157      build:
   158        context: ./app
   159      command: ["python", "worker.py"]
   160      environment:
   161        <<: *app-environment
   162        WORKER_POLL_SECONDS: ${WORKER_POLL_SECONDS}
   163      networks:
   164        - app-net
   165      depends_on:
   166        app-db:
   167          condition: service_healthy
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 2)

1. **Line 56-75** mengonfigurasi service log router `fluent-bit` berbasis image `fluent/fluent-bit:4.2.8`, port mapping host `24224` dan `2020` ke `127.0.0.1`, volume mapping disk buffer `fluent-bit-buffer` ke `/var/log/flb-storage`, serta deklarasi healthy dependency terhadap `log-db`.
2. **Line 76-95** mengonfigurasi service `auth-service` berbasis local build yang mengeksekusi Gunicorn server pada port internal `8081`, memvalidasi synthetic lab credentials, dan mengalirkan log ke Fluent Bit via logging driver.
3. **Line 96-115** mengonfigurasi service `registration-api` (Flask Web UI) pada port host `8080`, serta service `verification-worker` yang menjalankan worker asinkron `worker.py` dengan security options container `read_only: true` dan direktori sementara `tmpfs`.

Lampiran 2: init-app/01-init-app.sh (Inisialisasi Database Transactional PMB)

Skrip inisialisasi DDL SQL untuk membentuk role aplikasi, tabel transaction pendaftaran mahasiswa, dan kontrol mode insiden pada `app-db`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n init-app/01-init-app.sh
     1  #!/usr/bin/env sh
     2  set -eu
     3
     4  : "${POSTGRES_USER:?POSTGRES_USER belum diatur}"
     5  : "${POSTGRES_DB:?POSTGRES_DB belum diatur}"
     6  : "${APP_DB_USER:?APP_DB_USER belum diatur}"
     7  : "${APP_DB_PASSWORD:?APP_DB_PASSWORD belum diatur}"
     8
     9  psql \
    10    --set=ON_ERROR_STOP=1 \
    11    --username "$POSTGRES_USER" \
    12    --dbname "$POSTGRES_DB" \
    13    --set=app_user="$APP_DB_USER" \
    14    --set=app_password="$APP_DB_PASSWORD" <<'SQL'
    15  SELECT format('CREATE ROLE %I LOGIN PASSWORD %L', :'app_user', :'app_password')
    16  WHERE NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = :'app_user')
    17  \gexec
    18
    19  CREATE TABLE IF NOT EXISTS registrations (
    20      id BIGSERIAL PRIMARY KEY,
    21      public_id UUID NOT NULL UNIQUE,
    22      user_ref VARCHAR(64) NOT NULL,
    23      full_name VARCHAR(120) NOT NULL,
    24      email VARCHAR(255) NOT NULL UNIQUE,
    25      program VARCHAR(120) NOT NULL,
    26      status VARCHAR(20) NOT NULL
    27          CHECK (status IN ('SUBMITTED', 'PROCESSING', 'VERIFIED', 'FAILED')),
    28      attempt INTEGER NOT NULL DEFAULT 0 CHECK (attempt BETWEEN 0 AND 10),
    29      failure_code VARCHAR(64),
    30      request_id VARCHAR(128) NOT NULL,
    31      submitted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    32      updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
    33  );
    34
    35  CREATE INDEX IF NOT EXISTS idx_registrations_user_time
    36      ON registrations(user_ref, submitted_at DESC);
    37
    38  CREATE INDEX IF NOT EXISTS idx_registrations_status_time
    39      ON registrations(status, submitted_at);
    40
    41  CREATE INDEX IF NOT EXISTS idx_registrations_request
    42      ON registrations(request_id);
    43
    44  CREATE TABLE IF NOT EXISTS lab_control (
    45      key VARCHAR(64) PRIMARY KEY,
    46      value VARCHAR(64) NOT NULL,
    47      updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
    48  );
    49
    50  INSERT INTO lab_control(key, value)
    51  VALUES ('incident_mode', 'none')
    52  ON CONFLICT (key) DO NOTHING;
    53
    54  SELECT format('GRANT CONNECT ON DATABASE %I TO %I', current_database(), :'app_user')
    55  \gexec
    56  SELECT format('GRANT USAGE ON SCHEMA public TO %I', :'app_user')
    57  \gexec
    58  SELECT format('GRANT SELECT, INSERT, UPDATE ON TABLE registrations TO %I', :'app_user')
    59  \gexec
    60  SELECT format('GRANT SELECT ON TABLE lab_control TO %I', :'app_user')
    61  \gexec
    62  SELECT format('GRANT USAGE, SELECT ON SEQUENCE registrations_id_seq TO %I', :'app_user')
    63  \gexec
    64  SQL
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-10** mendefinisikan shebang shell, mode proteksi ketat `set -eu`, serta validasi keberadaan variabel lingkungan kredensial basis data transactional.
2. **Line 11-24** mengeksekusi pembuatan role `app_user` secara aman (*conditional DDL*) serta mendeklarasikan tabel `registrations` yang memuat kolom `public_id UUID`, `user_ref`, `full_name`, `email`, `status`, `attempt`, `failure_code`, dan `request_id`.
3. **Line 25-38** membangun indeks performa tinggi: indeks B-Tree pada `user_ref` dan waktu pendaftaran, indeks pada kolom `status`, serta indeks pada kolom `request_id` untuk akselerasi query korelasi.
4. **Line 39-55** membuat tabel kontrol pengujian `lab_control` untuk menyimpan nilai variabel `incident_mode` (default `none`), serta memberikan hak akses selektif (*least privilege*) kepada pengguna `app_user`.

Lampiran 3: init-log/01-init-log.sh (Inisialisasi Database Centralized Logging)

Skrip inisialisasi DDL SQL untuk membentuk skema telemetri, tabel log biner JSONB, dan view analitik pada `log-db`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n init-log/01-init-log.sh
     1  #!/usr/bin/env sh
     2  set -eu
     3
     4  : "${POSTGRES_USER:?POSTGRES_USER belum diatur}"
     5  : "${POSTGRES_DB:?POSTGRES_DB belum diatur}"
     6  : "${FLUENTBIT_DB_USER:?FLUENTBIT_DB_USER belum diatur}"
     7  : "${FLUENTBIT_DB_PASSWORD:?FLUENTBIT_DB_PASSWORD belum diatur}"
     8
     9  psql \
    10    --set=ON_ERROR_STOP=1 \
    11    --username "$POSTGRES_USER" \
    12    --dbname "$POSTGRES_DB" \
    13    --set=fb_user="$FLUENTBIT_DB_USER" \
    14    --set=fb_password="$FLUENTBIT_DB_PASSWORD" <<'SQL'
    15  SELECT format('CREATE ROLE %I LOGIN PASSWORD %L', :'fb_user', :'fb_password')
    16  WHERE NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = :'fb_user')
    17  \gexec
    18
    19  CREATE SCHEMA IF NOT EXISTS logs;
    20
    21  CREATE TABLE IF NOT EXISTS logs.fluentbit (
    22      tag TEXT NOT NULL,
    23      time TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    24      data JSONB NOT NULL
    25  );
    26
    27  CREATE INDEX IF NOT EXISTS idx_fluentbit_time
    28      ON logs.fluentbit(time DESC);
    29  CREATE INDEX IF NOT EXISTS idx_fluentbit_tag
    30      ON logs.fluentbit(tag);
    31  CREATE INDEX IF NOT EXISTS idx_fluentbit_data_gin
    32      ON logs.fluentbit USING GIN(data);
    33
    34  CREATE OR REPLACE VIEW logs.recent_logs AS
    35  SELECT
    36      time,
    37      tag,
    38      replace(data ->> 'container_name', '/', '') AS container_name,
    39      data ->> 'source' AS source,
    40      left(data ->> 'log', 500) AS log_preview
    41  FROM logs.fluentbit
    42  ORDER BY time DESC
    43  LIMIT 200;
    44
    45  SELECT format(
    46      'ALTER ROLE %I IN DATABASE %I SET search_path TO logs, public',
    47      :'fb_user', current_database()
    48  )
    49  \gexec
    50  SELECT format('GRANT USAGE, CREATE ON SCHEMA logs TO %I', :'fb_user')
    51  \gexec
    52  SELECT format('GRANT SELECT, INSERT ON TABLE logs.fluentbit TO %I', :'fb_user')
    53  \gexec
    54  SELECT format('GRANT SELECT ON TABLE logs.recent_logs TO %I', :'fb_user')
    55  \gexec
    56  SQL
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-10** mendefinisikan shebang shell, mode proteksi ketat, dan validasi variabel lingkungan akun log shipper `fb_user`.
2. **Line 11-25** membuat role `fb_user`, membentuk skema terisolasi `CREATE SCHEMA IF NOT EXISTS logs`, serta membuat tabel utama `CREATE TABLE logs.fluentbit (tag TEXT, time TIMESTAMP, data JSONB NOT NULL)`.
3. **Line 26-38** membangun indeks performa telemetri: indeks B-Tree pada `time DESC` dan `tag`, serta indeks inversi tergeneralisasi GIN pada kolom muatan `data` (`idx_fluentbit_data_gin`).
4. **Line 39-55** menyusun tampilan analitik `CREATE OR REPLACE VIEW logs.recent_logs` untuk audit visual cepat, serta menetapkan direktif `ALTER ROLE fb_user IN DATABASE log_db SET search_path TO logs, public` dan memberikan izin DML secara ketat.

Lampiran 4: fluent-bit/fluent-bit.conf (Konfigurasi Engine Pipeline Fluent Bit PMB)

Berkas konfigurasi engine Fluent Bit v4.2 yang mengatur penerimaan log Forward, buffering berbasis filesystem, dan output ke PostgreSQL.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n fluent-bit/fluent-bit.conf
     1  [SERVICE]
     2      Flush                     1
     3      Daemon                    Off
     4      Log_Level                 info
     5      storage.path              /var/log/flb-storage
     6      storage.sync              normal
     7      storage.checksum          on
     8      storage.backlog.mem_limit 10M
     9      HTTP_Server               On
    10      HTTP_Listen               0.0.0.0
    11      HTTP_Port                 2020
    12
    13  [INPUT]
    14      Name          forward
    15      Listen        0.0.0.0
    16      Port          24224
    17      Mem_Buf_Limit 10M
    18      storage.type  filesystem
    19
    20  [OUTPUT]
    21      Name                    pgsql
    22      Match                   pmb.*
    23      Host                    log-db
    24      Port                    5432
    25      User                    ${FLUENTBIT_DB_USER}
    26      Password                ${FLUENTBIT_DB_PASSWORD}
    27      Database                ${LOG_DB_NAME}
    28      Table                   fluentbit
    29      storage.total_limit_size 256M
    30
    31  [OUTPUT]
    32      Name   stdout
    33      Match  pmb.*
    34      Format json_lines
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-8** mengatur blok `[SERVICE]` dengan frekuensi flush 1 detik, level log info, dan aktivasi HTTP server internal pada port `2020` untuk pemantauan metrik.
2. **Line 9-15** mengaktifkan penyimpanan buffer antrean fisik: `storage.path /var/log/flb-storage`, sinkronisasi normal, batas memori backlog 5MB, serta toleransi metrik penyimpanan.
3. **Line 16-22** mengatur blok `[INPUT]` protokol Forward pada port `24224`, batas buffer memori 10MB, dan direktif krusial `storage.type filesystem` agar setiap log records langsung dicadangkan ke media disk.
4. **Line 23-32** mengatur blok `[OUTPUT]` plugin `pgsql` yang menyalurkan aliran log records ke host `log-db` port `5432`, tabel `fluentbit` (skema otomatis dari search_path), akun `fb_user`, serta pembatasan total antrean buffer di disk `storage.total_limit_size 256M`.

Lampiran 5: queries/incident-analysis.sql (Query Analisis SQL Investigasi Insiden)

Koleksi query SQL analitik untuk memverifikasi ketersediaan log, merekonstruksi kronologi kegagalan, dan mengaudit data sensitif.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n queries/incident-analysis.sql | head -n 35
     1  \pset pager off
     2  \timing on
     3
     4  -- 1. Memastikan log dari ketiga service tersedia.
     5  SELECT tag, count(*) AS total_event
     6  FROM logs.fluentbit
     7  GROUP BY tag
     8  ORDER BY total_event DESC, tag;
     9
    10  -- 2. Kronologi event aplikasi. Cast aman dibatasi pada record JSON buatan aplikasi.
    11  WITH events AS (
    12      SELECT time, tag, (data ->> 'log')::jsonb AS event
    13      FROM logs.fluentbit
    14      WHERE tag LIKE 'pmb.%'
    15        AND left(ltrim(data ->> 'log'), 1) = '{'
    16  )
    17  SELECT
    18      time,
    19      event ->> 'service' AS service,
    20      event ->> 'severity' AS severity,
    21      event ->> 'event' AS event_name,
    22      event ->> 'request_id' AS request_id,
    23      event ->> 'registration_id' AS registration_id,
    24      event ->> 'error_code' AS error_code
    25  FROM events
    26  ORDER BY time DESC
    27  LIMIT 50;
    28
    29  -- 3. Semua event WARN dan ERROR per service.
    30  WITH events AS (
    31      SELECT time, (data ->> 'log')::jsonb AS event
    32      FROM logs.fluentbit
    33      WHERE tag LIKE 'pmb.%'
    34        AND left(ltrim(data ->> 'log'), 1) = '{'
    35  )
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 1)

1. **Line 1-4** menonaktifkan interactive terminal pagination `\pset pager off` dan menyalakan penghitung durasi eksekusi query `\timing on`.
2. **Line 5-12** menjalankan Query 1 untuk mengelompokkan dan menghitung total log records yang diterima per tag sumber kontainer.
3. **Line 13-28** menjalankan Query 2 yang mengekstrak 50 event records aplikasi terbaru dengan membedah atribut JSONB: `service`, `severity`, `event`, `request_id`, dan `error_code`.
4. **Line 29-35** menjalankan Query 3 yang mengagregasikan seluruh event records berstatus `WARN` dan `ERROR` per layanan untuk early anomaly detection.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n queries/incident-analysis.sql | sed -n '36,70p'
    36  SELECT
    37      event ->> 'service' AS service,
    38      event ->> 'severity' AS severity,
    39      count(*) AS total
    40  FROM events
    41  WHERE event ->> 'severity' IN ('WARN', 'ERROR')
    42  GROUP BY service, severity
    43  ORDER BY total DESC, service;
    44
    45  -- 4. Ganti nilai request ID untuk merekonstruksi satu transaksi.
    46  \set investigated_request_id 'REQ-GANTI-DENGAN-NILAI-AKTUAL'
    47  WITH events AS (
    48      SELECT time, tag, (data ->> 'log')::jsonb AS event
    49      FROM logs.fluentbit
    50      WHERE tag LIKE 'pmb.%'
    51        AND left(ltrim(data ->> 'log'), 1) = '{'
    52  )
    53  SELECT
    54      time,
    55      event ->> 'service' AS service,
    56      event ->> 'event' AS event_name,
    57      event ->> 'severity' AS severity,
    58      event ->> 'attempt' AS attempt,
    59      event ->> 'error_code' AS error_code
    60  FROM events
    61  WHERE event ->> 'request_id' = :'investigated_request_id'
    62  ORDER BY time;
    63
    64  -- 5. Laju ingest per menit.
    65  SELECT date_trunc('minute', time) AS minute_bucket, count(*) AS total
    66  FROM logs.fluentbit
    67  GROUP BY minute_bucket
    68  ORDER BY minute_bucket DESC
    69  LIMIT 15;
    70
    71  -- 6. Pemeriksaan indikasi secret pada payload log.
    72  SELECT time, tag, left(data ->> 'log', 300) AS suspicious_message
    73  FROM logs.fluentbit
    74  WHERE data ->> 'log' ~* '(password|passwd|authorization|bearer|cookie|session[_-]?secret|api[_-]?key)'
    75  ORDER BY time DESC;
    76
    77  -- 7. Preview data sebelum retensi. Jangan hapus tanpa review.
    78  SELECT count(*) AS candidate_for_deletion
    79  FROM logs.fluentbit
    80  WHERE time < CURRENT_TIMESTAMP - INTERVAL '7 days';
    81
    82  -- DELETE FROM logs.fluentbit
    83  -- WHERE time < CURRENT_TIMESTAMP - INTERVAL '7 days';
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 2)

1. **Line 36-48** mendefinisikan Query 4 berbasis parameter `investigated_request_id` untuk merekonstruksi transaction lifecycle secara kronologis mulai dari pembuatan hingga penyelesaian.
2. **Line 49-55** menjalankan Query 5 yang mengukur laju penyerapan log per menit (*minute-bucket ingest rate*) pada tabel `logs.fluentbit`.
3. **Line 56-63** menjalankan Query 6 untuk mengaudit potensi kebocoran data sensitif (*PII and secret scanning*) menggunakan pola regex terhadap kata sandi dan token.
4. **Line 64-70** menjalankan Query 7 untuk melakukan preview terhadap data kandidat penghapusan sesuai kebijakan data retention berbasis umur record lebih dari 7 hari.

Lampiran 6: scripts/demo-normal.sh (Skrip Otomatisasi Transaction Normal)

Skrip bash untuk mengeksekusi skenario pendaftaran normal dan memvalidasi transisi status menjadi `VERIFIED`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n scripts/demo-normal.sh
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5  source scripts/lib.sh
     6  load_env
     7
     8  wait_http "Portal PMB" "http://127.0.0.1:8080/healthz"
     9  ./scripts/set-incident.sh none >/dev/null
    10  request_id="$(submit_application NORMAL)"
    11  echo "[INFO] Request ID: $request_id"
    12  sleep 5
    13
    14  docker compose exec -T app-db psql \
    15      --username "$APP_DB_ADMIN_USER" --dbname "$APP_DB_NAME" \
    16      --set=request_id="$request_id" <<'SQL'
    17  SELECT public_id, status, attempt, failure_code, request_id
    18  FROM registrations
    19  WHERE request_id = :'request_id';
    20  SQL
    21
    22  echo "[PASS] Demo normal selesai. Status yang diharapkan: VERIFIED."
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-6** mendefinisikan shebang bash, mode proteksi ketat `set -Eeuo pipefail`, memuat helper library `scripts/lib.sh`, serta mengaktifkan environment variables dari file `.env`.
2. **Line 7-12** menunggu hingga endpoint portal PMB berstatus healthy, menyetel incident mode ke `none`, dan memanggil fungsi `submit_application NORMAL` yang mengembalikan token `request_id` unik.
3. **Line 13-21** mengeksekusi query psql ke `app-db` untuk memverifikasi bahwa status pendaftaran telah berhasil diverifikasi (`status = 'VERIFIED'`).

Lampiran 7: scripts/demo-incident.sh (Skrip Otomatisasi Simulasi Insiden)

Skrip bash untuk menginjeksi simulasi kegagalan dependensi timeout dan menghasilkan transaction berstatus `FAILED`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n scripts/demo-incident.sh
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5  source scripts/lib.sh
     6  load_env
     7
     8  wait_http "Portal PMB" "http://127.0.0.1:8080/healthz"
     9  ./scripts/set-incident.sh timeout >/dev/null
    10  request_id="$(submit_application INCIDENT)"
    11  echo "[INFO] Request ID: $request_id"
    12  sleep 8
    13
    14  docker compose exec -T app-db psql \
    15      --username "$APP_DB_ADMIN_USER" --dbname "$APP_DB_NAME" \
    16      --set=request_id="$request_id" <<'SQL'
    17  SELECT public_id, status, attempt, failure_code, request_id
    18  FROM registrations
    19  WHERE request_id = :'request_id';
    20  SQL
    21
    22  echo "[INFO] Gunakan request ID di atas pada queries/incident-analysis.sql."
    23  echo "[PASS] Demo insiden selesai. Status yang diharapkan: FAILED / VERIFY-TIMEOUT."
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-6** mendefinisikan shebang bash, mode penanganan pipeline error ketat `set -Eeuo pipefail`, serta pemuatan helper library pendukung.
2. **Line 7-12** mengubah incident mode pada database ke `timeout`, mengirim pendaftaran baru, dan mencatat nilai `request_id` insiden untuk pelacakan.
3. **Line 13-22** menunggu hingga worker menyelesaikan siklus 3x retry (interval 8 detik) dan mengeksekusi query psql untuk mengonfirmasi transisi status akhir menjadi `FAILED / VERIFY-TIMEOUT`.

Lampiran 8: scripts/collect-evidence.sh (Skrip Pengumpulan Bukti Operasional)

Skrip otomatisasi untuk mengompilasi seluruh bukti operasional, status telemetri, dan menghitung checksum SHA-256 berkas konfigurasi ke dalam direktori `reports/`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n scripts/collect-evidence.sh
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5  source scripts/lib.sh
     6  load_env
     7  mkdir -p reports
     8
     9  docker compose ps >reports/compose-ps.txt
    10  curl -fsS http://127.0.0.1:2020/api/v1/metrics >reports/fluent-bit-metrics.json
    11
    12  docker compose exec -T app-db psql \
    13      --username "$APP_DB_ADMIN_USER" --dbname "$APP_DB_NAME" \
    14      --command='SELECT public_id, status, attempt, failure_code, request_id, updated_at FROM registrations ORDER BY submitted_at;' \
    15      >reports/registration-status.txt
    16
    17  docker compose exec -T log-db psql \
    18      --username "$LOG_DB_ADMIN_USER" --dbname "$LOG_DB_NAME" \
    19      --file=/dev/stdin <queries/incident-analysis.sql \
    20      >reports/incident-analysis.txt
    21
    22  sha256sum compose.yaml app/*.py app/requirements.txt fluent-bit/fluent-bit.conf \
    23      init-app/01-init-app.sh init-log/01-init-log.sh queries/incident-analysis.sql \
    24      >reports/config-sha256.txt
    25
    26  echo "[PASS] Evidence dibuat di direktori reports/. File .env tidak disertakan."
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-6** mendefinisikan interpreter bash, inisialisasi environment variables, serta pembuatan direktori output artefak bukti `reports/`.
2. **Line 7-14** mengekspor status kontainer Docker Compose, mengambil metrik internal Fluent Bit via HTTP, dan mengekstrak tabel pendaftaran bisnis dari `app-db` tanpa menyertakan kolom PII.
3. **Line 15-25** mengeksekusi query investigasi SQL ke `log-db`, menghitung hash kriptografis SHA-256 seluruh berkas konfigurasi, serta mencetak konfirmasi bahwa berkas rahasia `.env` aman dan tidak disertakan.

Catatan & Pernyataan Penggunaan AI:

Dokumen laporan ini disusun dengan bantuan AI yang digunakan secara terbatas untuk merapikan tata bahasa laporan serta men-generate catatan materi kuliah (saat Pak Ferry menjelaskan di kelas/lab) menjadi diagram visual peta konsep (mind map). Seluruh jawaban pada bagian Evaluasi dan Latihan Mandiri, analisis teknis, serta pengujian praktikum merupakan hasil pengerjaan, pemahaman mandiri, dan analisis saya secara langsung.